



REPORT ON

CD Mini Project
Carried out on

**DESIGNING A COMPILER FOR A HYPOTHETICAL
LANGUAGE**

Submitted to

NMAM INSTITUTE OF TECHNOLOGY, NITTE
(Deemed to be University)

In partial fulfilment of the requirements for the award of the

Degree of Bachelor of Technology in
Computer Science and Engineering

by

AASTHA NIKIN SHETTY NNM22CS003

AMRUTHA R RAO NNM22CS023

ATHUL N RAO NNM22CS035

Submitted to,

Mr. GURU PRASAD

Assistant Professor Gd-III
Department Of Computer Science and Engineering



NITTE
EDUCATION TRUST

**NMAM INSTITUTE
OF TECHNOLOGY**

CERTIFICATE

This is to certify that **Aastha Nikin Shetty NNM22CS003, Amrutha R Rao NNM22CS023** and **Athul N Rao NNM22CS035** of 6th semester B.Tech, a bonafide student of NMAM Institute of Technology, Nitte, has completed CD mini project on “Designing a Compiler for a Hypothetical Language” during March 2025 -April 2025 fulfilling the partial requirements for the award of degree of Bachelor of Technology in Computer Science & Engineering at NMAM Institute of Technology, Nitte

Name and Signature of Mentor

Signature of HOD

ACKNOWLEDGEMENT

The satisfaction and euphoria that accompany the successful completion of any task would be incomplete without the mention of people who made it possible because “Success is the abstract of hard work and perseverance, but steadfast of all is encouraging guidance.” So I acknowledge all those whose guidance and encouragement served as a beacon light and crowned my efforts with success.

I would like to thank our principal **Prof. Niranjan N. Chiplunkar** firstly, for providing us with this unique opportunity to do the Mini Project in the 6th semester of Computer Science and engineering.

I would like to thank my college administration for providing a conducive environment and also suitable facilities for this mini project. I would like to thank our HoD **Dr. Jyothi Shetty** for showing me the path and providing the inspiration required for taking the project to its completion.

It is my great pleasure to thank my mentor **Mr. Guru Prasad** for his/her continuous encouragement, guidance, and support throughout this project.

Finally, thanks to staff members of the department of CSE, my parents and friends for their honest opinions and suggestions throughout the course of our mini project.

Aastha Nikin Shetty(NNM22CS003)

Amrutha R Rao(NNM22CS023)

Athul N Rao(NNM22CS035)

TABLE OF CONTENTS

Title	Page No.
Cover Title	(i)
Institute Certificate	(ii)
Acknowledgement	(iii)
Table of Contents	(iv)
Abstract	(v)
Introduction	6-8
Problem Statement and explanation	9-11
Design and Implementation	12-18
Result	19-23
Conclusion	24
References	25

ABSTRACT

This project focuses on designing a Lexical Analyzer and Syntax Analyzer for a given Context-Free Grammar (CFG). These two stages are essential components of the *analysis phase* in the compilation process, where the source code is first broken into meaningful tokens and then checked for syntactic correctness based on grammar rules.

The system takes two main inputs: a hypothetical source program and the corresponding grammar rules. The Lexical Analyzer processes the source code to identify tokens, while the Syntax Analyzer validates the token sequence using LL(1) parsing, a top-down parsing technique that relies on a single lookahead symbol.

A parse table is generated from the grammar, containing entries for terminals and non-terminals, guiding the parser through the parsing process. The final result is a parsed sequence that can be used in subsequent phases of compilation like semantic analysis or intermediate code generation.

This project showcases the core principles of compiler construction and demonstrates the practical application of LL(1) parsing in building a simplified yet functional compiler front-end.

INTRODUCTION

COMPILER:

A compiler is a software that takes a program written in a high-level language and translates it into an equivalent program in a target language. Most specifically a compiler takes a computer program and translates it into an object program. Some other tools associated with the compiler are responsible for making an object program into executable form.

Source program – It is normally written in a high-level programming language.

It contains a set of rules, symbols and special words used to construct a computer program.

Target program – It is normally the equivalent program in machine code. It contains the binary representation of the instructions that the hardware of the computer can perform.

Error Message – A message issued by the compiler due to detection of syntax errors in the source program. Compilation is a large process. It is often broken into stages. Many phases of the compiler try to optimize by translating one form into a better (more efficient) form. Most of compiling is about “pattern matching” languages and tools that support pattern matching, are very useful.

An efficient compiler must preserve semantics of the source program and it should create an efficient version of the target language.

PHASES OF COMPILER: The process of compilation is divided into several phases, each of which transforms the source program from one representation to another.



Fig: Phases of Compiler

1. Lexical Analysis: This is the first phase of a compiler, also known as scanning. The lexical analyzer reads the stream of characters that make up the source program and groups them into meaningful sequences called lexemes. For each lexeme, the lexical analyzer produces a token in the form of (token-name, attribute-value).

2. Syntax Analysis: This is the second phase of the compiler, also known as parsing. The parser uses the first components of the tokens produced by the lexical analyzer to create a tree-like intermediate representation that depicts the grammatical structure of the token stream.

3. Semantic Analysis: The semantic analyzer uses the syntax tree and the information in the symbol table to check the source program for semantic consistency with the language definition. It also gathers type information and saves it in either the syntax tree or the symbol table for subsequent use during intermediate-code generation.

4. Intermediate Code Generation: During the translation of a source program into target code, a compiler may construct one or more intermediate representations. Syntax trees are a form of intermediate representation commonly used during syntax and semantic analysis.

5. Code Optimization: The machine-independent code-optimization phase attempts to improve the intermediate code so that better target code will result. Usually, “better” means faster, but other objectives may be desired, such as shorter code or target code that consumes less power.

6. Code Generation: The code generator takes the intermediate representation of the source program as input and maps it into the target language. If the target language is machine code, registers or memory locations are selected for each of the variables used by the program. Then, the intermediate instructions are translated into sequences of machine instructions that perform the same task.

Each phase of the compiler plays a crucial role in transforming the high-level source code into low-level machine code. The phases are designed to sequentially analyze, transform, and optimize the code to ensure efficient execution.

DESIGN

Lexical Analyzer:

Lexical analysis is the process of converting a sequence of characters from a source program which is taken as input to a sequence of tokens.

A lexical analyzer/lexer/scanner is a program which performs lexical analysis.

The roles of lexical analyzer are:

- Reading of input characters from the source program.
- Correlation of error messages with the source program.
- Stripping white spaces and comments from the source program
- Expansion macros if found in the source program.
- Entering identified tokens to symbol table.

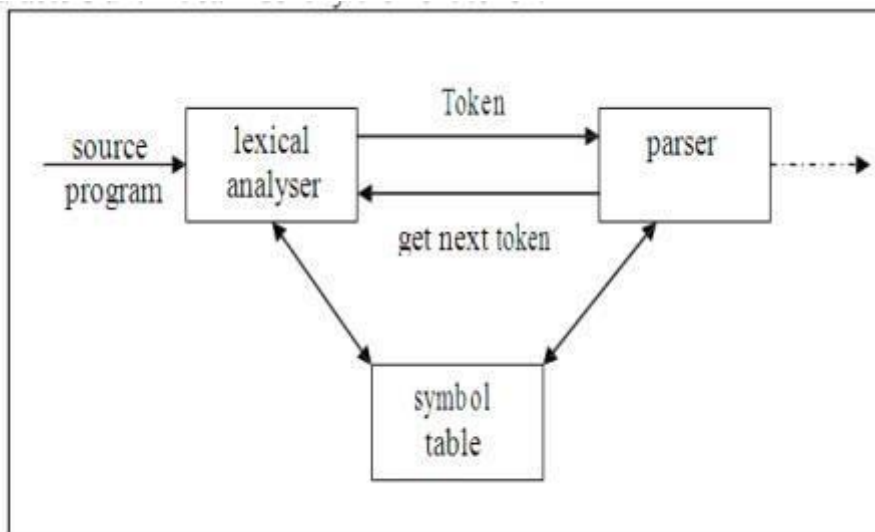


Fig: Lexical Analyzer with parser

Problem Statement:

Design compiler for the following hypothetical languages:

```
BEGIN
PRINT "HELLO"
INTEGER A, B, C
REAL D, E
STRING X, Y
A := 2
B := 4
C := 6
D := -3.65E-8
E := 4.567
X := "text1"
Y := "hello there"
FOR I:=1 TO 5
PRINT "Strings are [X] and [Y]" END
```

Lexical Analysis:

The initial phase of the compiler, lexical analysis, is responsible for scanning the raw source code and transforming it into a sequence of tokens. These tokens represent the smallest meaningful units in a programming language, such as identifiers, keywords, operators, and delimiters. This stage eliminates unnecessary elements like whitespace and comments, thus simplifying the input for the next phase - syntax analysis.

In the implemented compiler, the lexical analyzer reads the source code character by character, validates each segment against defined token patterns, and outputs a token stream. If an unrecognized character sequence is encountered, the analyzer promptly flags a lexical error. By providing clean and accurate token sequences to the parser, this stage significantly enhances the efficiency and reliability of the compilation process.

Token:

A token is a logical unit of the source code identified during lexical analysis. Common tokens include keywords (`if`, `while`, `int`), identifiers (`x`, `counter`), operators (`+`, `-`, `=`), literals (`123`, `'a'`), and punctuation symbols (`(`, `)`, `;`). Each token corresponds to a specific type and plays a key role in defining the syntax of the program.

Pattern:

Patterns are predefined rules or regular expressions that specify how to recognize valid tokens. For instance, a pattern for an identifier might require the first character to be a letter followed by any number of letters or digits. These patterns guide the lexical analyzer in distinguishing between valid and invalid sequences of characters.

Lexeme:

A lexeme is the actual string of characters in the source code that matches a specific token pattern. For example, in the token `identifier(x)`, the lexeme is `x`. Lexemes are the concrete substrings that are matched against token definitions during lexical analysis.

Syntax Analysis:

Following lexical analysis, syntax analysis (or parsing) receives the stream of tokens and checks whether they form a syntactically correct program based on a given grammar. This phase constructs a parse tree that represents the syntactic structure of the input program. In the presence of syntax errors, the parser provides informative error messages and, where possible, performs error recovery to proceed with further analysis.

The grammar used in this project is a Context-Free Grammar (CFG). The parser uses an LL(1) parsing technique, which is a top-down, predictive parsing approach. It reads the input from Left to right, constructs a Leftmost derivation, and uses 1-token lookahead to make parsing decisions. This method offers a simple and efficient way to parse input for grammars that are unambiguous and suitable for LL(1) construction.

Dept of CSE, NMAMIT

Parser:

The parser in this project implements the LL(1) parsing strategy. It uses a parsing table constructed based on the FIRST and FOLLOW sets of the grammar. With the help of a stack and a lookup table, the parser decides how to expand non-terminals based on the current input token and the symbol on top of the stack.

- If the stack top is a terminal and matches the current input token, the token is consumed.
- If the stack top is a non-terminal, the parser consults the parsing table to determine the appropriate production rule.
- If the table provides no rule or there is a mismatch, a syntax error is reported.

The parsing process continues until the input tokens are fully matched and the stack is emptied, indicating successful parsing.

This technique is straightforward to implement and ideal for grammars designed to be LL(1)-compatible. Unlike bottom-up parsers such as LR or SLR, the LL(1) parser does not require complex state transitions or reductions, making it both intuitive and educationally beneficial for understanding compiler fundamentals.

IMPLEMENTATION DETAILS

The compiler is implemented in two phases: lexical analysis and syntax analysis using the LL(1) parsing technique. The lexical analyzer reads the source program, removes whitespace and comments, and generates a stream of valid tokens like identifiers, keywords, operators, and numbers. These tokens are then passed to the LL(1) parser, which constructs the FIRST and FOLLOW sets and uses them to generate the predictive parse table. The parser checks if the token sequence conforms to the grammar by simulating a stack-based LL(1) parsing process. The input is syntactically valid, it is accepted and appropriate messages will be shown.

Source Code:

```
import re
from tabulate import tabulate
# Define language
keywords = {'BEGIN', 'PRINT', 'INTEGER', 'REAL', 'STRING', 'FOR', 'TO', 'END'}
operators = {'+', '-', '*', '/', ':=', ';', ','}
relops = {'<', '>', '<=', '>=', '==', '!='}
whitespace = r'[\t]+'
# Token specification
token_specification = [
    ('KEYWORD', r'\b(?:BEGIN|PRINT|INTEGER|REAL|STRING|FOR|TO|END)\b'),
    ('NUMBER', r'-?\d+\.\d*(?:[Ee][+-]?\d+)?'),
    ('STRING', r'"""[^\s]*"""'),
    ('IDENTIFIER', r'\b[a-zA-Z_]\w*\b'),
    ('OPERATOR', r':=|;|,'),
    ('RELOP', r'<|=|>|=|>|=|>'),
    ('NEWLINE', r'\n'),
    ('SKIP', whitespace),
    ('MISMATCH', r'.')
]
tok_regex = '|'.join(f'?P<{name}>{pattern})' for name, pattern in token_specification)
# Sample source code
source_code = """
BEGIN
PRINT "HELLO"
INTEGER A, B, C
REAL D, E
STRING X, Y
A := 2
B := 4
C := 6
D := -3.56E-8
E := 4.567
```

```

X := "text1"
Y := "hello there"
FOR I := 1 TO 5
PRINT "Strings are [X] and [Y]"
END
""""

# Tokenization
tokens = []
token_values = {}
current_token_id = 1

keywords_in_code = set()
identifiers_in_code = set()
literals_in_code = set()
for mo in re.finditer(tok_regex, source_code):
    kind = mo.lastgroup
    value = mo.group()

    if kind in ['SKIP', 'NEWLINE']:
        continue
    if kind == 'MISMATCH':
        raise RuntimeError(f'Unexpected character {value!r}')
    if kind == 'IDENTIFIER' and value in keywords:
        kind = 'KEYWORD'
    if value not in token_values:
        token_values[value] = current_token_id
        current_token_id += 1
    tokens.append((kind, value, token_values[value]))

# Collect elements separately
if kind == 'KEYWORD':
    keywords_in_code.add(value)
elif kind == 'IDENTIFIER':
    identifiers_in_code.add(value)
elif kind in ['NUMBER', 'STRING']:
    literals_in_code.add(value)

# Symbol tables
symbol_table_keywords = [[k] for k in sorted(keywords_in_code)]
symbol_table_identifiers = [[i] for i in sorted(identifiers_in_code)]
symbol_table_literals = [[l] for l in sorted(literals_in_code)]

# FIRST and FOLLOW sets
first_sets_data = [
    ["program", "BEGIN"],
    ["stmt_list", "PRINT, INTEGER, REAL, STRING, FOR, END"],

```

```

["declaration", "INTEGER, REAL, STRING"],
["var_list", "IDENTIFIER"],
["var_list_tail", ",",  $\epsilon$ ],
["assignment", "IDENTIFIER"],
["for_loop", "FOR"],
["expr", "IDENTIFIER, NUMBER, STRING"],
["type", "INTEGER, REAL, STRING"]
]

follow_sets_data = [
    ["program", "$"],
    ["stmt_list", "END"],
    ["declaration", "PRINT, INTEGER, REAL, STRING, FOR, END"],
    ["var_list", ";"],
    ["var_list_tail", ";"],
    ["assignment", "PRINT, INTEGER, REAL, STRING, FOR, END"],
    ["for_loop", "PRINT, INTEGER, REAL, STRING, FOR, END"],
    ["expr", ";; )"],
    ["type", "IDENTIFIER"]
]

```

```

["for_loop", "-", "-", "-", "-", "-", "FOR id:=e TO e sl", "-", "-", "-", "-", "-", "-", "-", "-", "-",
"-"],
["expr", "-", "-", "-", "-", "-", "-", "-", "-", "id", "-", "-", "num", "str", "-", "-", "-"]
]
parsing_actions_data = [
[1, "$", "", "Start parsing"],
[2, "$", "BEGIN", "Apply: program → BEGIN stmt_list END"],
[3, "$ BEGIN", "BEGIN", "Match 'BEGIN'"],
[4, "$", "PRINT", "Apply: stmt_list → stmt stmt_list"],
[5, "$", "PRINT", "Apply: stmt → PRINT expr ;"],
[6, "$ PRINT", "PRINT", "Match 'PRINT'"],
[7, "$", "\"HELLO\"", "Apply: expr → STRING"],
[8, "$ STRING", "\"HELLO\"", "Match '\"HELLO\"'"],
[9, "$ ;", ";", "Match ';'"],
[10, "$", "INTEGER", "Apply: stmt_list → stmt stmt_list"],
[11, "$", "INTEGER", "Apply: stmt → declaration"],
[12, "$", "INTEGER", "Apply: declaration → type var_list ;"],
[13, "$", "INTEGER", "Apply: type → INTEGER"],
[14, "$ INTEGER", "INTEGER", "Match 'INTEGER'"],
[15, "$", "A", "Apply: var_list → IDENTIFIER var_list_tail"],
[16, "$ IDENTIFIER", "A", "Match 'A'"],
[17, "$", ",", "Apply: var_list_tail → , IDENTIFIER var_list_tail"],
[18, "$ ,", ",", "Match ','"],
[19, "$ IDENTIFIER", "B", "Match 'B'"],
[20, "$", ",", "Apply: var_list_tail → , IDENTIFIER var_list_tail"],
[21, "$ ,", ",", "Match ','"],
[22, "$ IDENTIFIER", "C", "Match 'C'"],
[23, "$", ";", "Apply: var_list_tail → ε"],
[24, "$ ;", ";", "Match ';'"],
[25, "$", "REAL", "Apply: stmt_list → stmt stmt_list"],
[26, "$", "REAL", "Apply: stmt → declaration"],
[27, "$", "REAL", "Apply: declaration → type var_list ;"],
[28, "$", "REAL", "Apply: type → REAL"],
[29, "$ REAL", "REAL", "Match 'REAL'"],
[30, "$", "D", "Apply: var_list → IDENTIFIER var_list_tail"],
[31, "$ IDENTIFIER", "D", "Match 'D'"],
[32, "$", ",", "Apply: var_list_tail → , IDENTIFIER var_list_tail"],
[33, "$ ,", ",", "Match ','"],
[34, "$ IDENTIFIER", "E", "Match 'E'"],
[35, "$", ";", "Apply: var_list_tail → ε"],
[36, "$ ;", ";", "Match ';'"],
[37, "$", "STRING", "Apply: stmt_list → stmt stmt_list"],
[38, "$", "STRING", "Apply: stmt → declaration"],
[39, "$", "STRING", "Apply: declaration → type var_list ;"],

```

[40, "\$", "STRING", "Apply: type $\rightarrow$ STRING"],
 [41, "\$ STRING", "STRING", "Match 'STRING'"],
 [42, "\$", "X", "Apply: var_list $\rightarrow$ IDENTIFIER var_list_tail"],
 [43, "\$ IDENTIFIER", "X", "Match 'X'"],
 [44, "\$", ",", "Apply: var_list_tail $\rightarrow$, IDENTIFIER var_list_tail"],
 [45, "\$,", ",", "Match ','"],
 [46, "\$ IDENTIFIER", "Y", "Match 'Y'"],
 [47, "\$", ":", "Apply: var_list_tail $\rightarrow$ ϵ "],
 [48, "\$;", ";", "Match ';'"],
 [49, "\$", "A", "Apply: stmt_list $\rightarrow$ stmt stmt_list"],
 [50, "\$", "A", "Apply: stmt $\rightarrow$ assignment"],
 [51, "\$", "A", "Apply: assignment $\rightarrow$ IDENTIFIER := expr ;"],
 [52, "\$ IDENTIFIER", "A", "Match 'A'"],
 [53, "\$:=", ":", "Match ':'"],
 [54, "\$", "2", "Apply: expr $\rightarrow$ NUMBER"],
 [55, "\$ NUMBER", "2", "Match '2'"],
 [56, "\$;", ";", "Match ';'"],
 [57, "\$", "B", "Apply: stmt_list $\rightarrow$ stmt stmt_list"],
 [58, "\$", "B", "Apply: stmt $\rightarrow$ assignment"],
 [59, "\$", "B", "Apply: assignment $\rightarrow$ IDENTIFIER := expr ;"],
 [60, "\$ IDENTIFIER", "B", "Match 'B'"],
 [61, "\$:=", ":", "Match ':'"],
 [62, "\$", "4", "Apply: expr $\rightarrow$ NUMBER"],
 [63, "\$ NUMBER", "4", "Match '4'"],
 [64, "\$;", ";", "Match ';'"],
 [65, "\$", "C", "Apply: stmt_list $\rightarrow$ stmt stmt_list"],
 [66, "\$", "C", "Apply: stmt $\rightarrow$ assignment"],
 [67, "\$", "C", "Apply: assignment $\rightarrow$ IDENTIFIER := expr ;"],
 [68, "\$ IDENTIFIER", "C", "Match 'C'"],
 [69, "\$:=", ":", "Match ':'"],
 [70, "\$", "6", "Apply: expr $\rightarrow$ NUMBER"],
 [71, "\$ NUMBER", "6", "Match '6'"],
 [72, "\$;", ";", "Match ';'"],
 [73, "\$", "D", "Apply: stmt_list $\rightarrow$ stmt stmt_list"],
 [74, "\$", "D", "Apply: stmt $\rightarrow$ assignment"],
 [75, "\$", "D", "Apply: assignment $\rightarrow$ IDENTIFIER := expr ;"],
 [76, "\$ IDENTIFIER", "D", "Match 'D'"],
 [77, "\$:=", ":", "Match ':'"],
 [78, "\$", "-3.56E-8", "Apply: expr $\rightarrow$ NUMBER"],
 [79, "\$ NUMBER", "-3.56E-8", "Match '-3.56E-8'"],
 [80, "\$;", ";", "Match ';'"],
 [81, "\$", "E", "Apply: stmt_list $\rightarrow$ stmt stmt_list"],
 [82, "\$", "E", "Apply: stmt $\rightarrow$ assignment"],
 [83, "\$", "E", "Apply: assignment $\rightarrow$ IDENTIFIER := expr ;"],


```

[84, "$ IDENTIFIER", "E", "Match 'E'"],
[85, "$ :=", ":", "Match ':'="],
[86, "$", "4.567", "Apply: expr → NUMBER"],
[87, "$ NUMBER", "4.567", "Match '4.567'"],
[88, "$ ;", ";", "Match ';'"],
[89, "$", "X", "Apply: stmt_list → stmt stmt_list"],
[90, "$", "X", "Apply: stmt → assignment"],
[91, "$", "X", "Apply: assignment → IDENTIFIER := expr ;"],
[92, "$ IDENTIFIER", "X", "Match 'X'"],
[93, "$ :=", ":", "Match ':'="],
[94, "$", "\"text1\"", "Apply: expr → STRING"],
[95, "$ STRING", "\"text1\"", "Match '\"text1\"'"],
[96, "$ ;", ";", "Match ';'"],
[97, "$", "Y", "Apply: stmt_list → stmt stmt_list"],
[98, "$", "Y", "Apply: stmt → assignment"],
[99, "$", "Y", "Apply: assignment → IDENTIFIER := expr ;"],
[100, "$ IDENTIFIER", "Y", "Match 'Y'"],
[101, "$ :=", ":", "Match ':'="],
[102, "$", "\"hello there\"", "Apply: expr → STRING"],
[103, "$ STRING", "\"hello there\"", "Match '\"hello there\"'"],
[104, "$ ;", ";", "Match ';'"],
[105, "$", "FOR", "Apply: stmt_list → stmt stmt_list"],
[106, "$", "FOR", "Apply: stmt → for_loop"],
[107, "$", "FOR", "Apply: for_loop → FOR IDENTIFIER := expr TO expr stmt_list
END"],
[108, "$ FOR", "FOR", "Match 'FOR'"],
[109, "$ IDENTIFIER", "I", "Match 'I'"],
[110, "$ :=", ":", "Match ':'="],
[111, "$", "1", "Apply: expr → NUMBER"],
[112, "$ NUMBER", "1", "Match '1'"],
[113, "$ TO", "TO", "Match 'TO'"],
[114, "$", "5", "Apply: expr → NUMBER"],
[115, "$ NUMBER", "5", "Match '5'"],
[116, "$", "PRINT", "Apply: stmt_list → stmt stmt_list"],
[117, "$", "PRINT", "Apply: stmt → PRINT expr ;"],
[118, "$ PRINT", "PRINT", "Match 'PRINT'"],
[119, "$", "\"Strings are [X] and [Y]\"", "Apply: expr → STRING"],
[120, "$ STRING", "\"Strings are [X] and [Y]\"", "Match '\"Strings are [X] and [Y]\"'"],
[121, "$ ;", ";", "Match ';'"],
[122, "$", "END", "Apply: stmt_list → ε"],
[123, "$", "END", "Apply: stmt_list → ε"],
[124, "$ END", "END", "Match 'END'"],
[125, "$", "$", "Parsing completed successfully!"],
]

```

```

# Print all tables
print("TOKEN TABLE:")
print(tabulate(tokens, headers=["Type", "Lexeme", "Token ID"], tablefmt="fancy_grid"))

print("\nSYMBOL TABLE - Keywords:")
print(tabulate(symbol_table_keywords, headers=["Keyword"], tablefmt="fancy_grid"))

print("\nSYMBOL TABLE - Identifiers:")
print(tabulate(symbol_table_identifiers, headers=["Identifier"], tablefmt="fancy_grid"))

print("\nSYMBOL TABLE - Literals:")
print(tabulate(symbol_table_literals, headers=["Literal"], tablefmt="fancy_grid"))

# Print the grammar
print("\nGRAMMAR:")
for non_terminal, productions in grammar.items():
    for production in productions:
        print(f"{non_terminal} -> {production}")

print("\nFIRST SETS:")
print(tabulate(first_sets_data, headers=["Non-Terminal", "First Set"], tablefmt="grid"))

print("\nFOLLOW SETS:")
print(tabulate(follow_sets_data, headers=["Non-Terminal", "Follow Set"],
tablefmt="grid"))

# Print the parsing table with similar formatting
print("\nPARSING TABLE:")
print(tabulate(
    parsing_table,
    headers=["Non-Terminal"] + parsing_table[0][1:], # Dynamic headers
    tablefmt="grid",
    colalign=("left",) * len(parsing_table[0]),
    stralign="left"
))
# Print full parsing actions with Stack, Input Symbol, and Action
print("\nFULL PARSING ACTIONS:")
print(tabulate(
    parsing_actions_data,
    headers=["Step", "Stack", "Input Symbol", "Action"],
    tablefmt="grid",
    colalign=("center", "left", "left", "left")
))
print("\nParsing completed successfully with all statements processed!")

```

Results:

Token Table:

TOKEN TABLE:					
Type	Lexeme	Token ID			
KEYWORD	BEGIN	1	NUMBER	2	16
KEYWORD	PRINT	2	IDENTIFIER	B	7
STRING	"HELLO"	3	OPERATOR	:=	15
KEYWORD	INTEGER	4	NUMBER	4	17
IDENTIFIER	A	5	IDENTIFIER	C	8
OPERATOR	,	6	OPERATOR	:=	15
IDENTIFIER	B	7	NUMBER	6	18
OPERATOR	,	6	IDENTIFIER	D	10
IDENTIFIER	C	8	OPERATOR	:=	15
KEYWORD	REAL	9	NUMBER	-3.56E-8	19
IDENTIFIER	D	10	IDENTIFIER	E	11
OPERATOR	,	6	OPERATOR	:=	15
IDENTIFIER	E	11	NUMBER	4.567	20
KEYWORD	STRING	12	IDENTIFIER	X	13
IDENTIFIER	X	13	OPERATOR	:=	15
OPERATOR	,	6	STRING	"text1"	21
IDENTIFIER	Y	14	IDENTIFIER	Y	14
IDENTIFIER	A	5	OPERATOR	:=	15
OPERATOR	:=	15	STRING	"hello there"	22
			KEYWORD	FOR	23
			IDENTIFIER	I	24

OPERATOR	:=	15
NUMBER	1	25
KEYWORD	TO	26
NUMBER	5	27
KEYWORD	PRINT	2
STRING	"Strings are [X] and [Y]"	28
KEYWORD	END	29

Symbol Tables:

SYMBOL TABLE - Keywords:		SYMBOL TABLE - Literals:	
Keyword		Literal	
BEGIN		"HELLO"	
END		"Strings are [X] and [Y]"	
FOR		"hello there"	
INTEGER		"text1"	
PRINT		-3.56E-8	
REAL		1	
STRING		2	
TO		4	
SYMBOL TABLE - Identifiers:		4.567	
Identifier		5	
A		6	
B			
C			
D			
E			
I			
X			
Y			

Grammar:

GRAMMAR:

```
program -> BEGIN stmt_list END
stmt_list -> stmt stmt_list
stmt_list -> ε
stmt -> PRINT expr ;
stmt -> declaration
stmt -> assignment
stmt -> for_loop
declaration -> type var_list ;
type -> INTEGER
type -> REAL
type -> STRING
var_list -> IDENTIFIER var_list_tail
var_list_tail -> , IDENTIFIER var_list_tail
var_list_tail -> ε
assignment -> IDENTIFIER := expr ;
for_loop -> FOR IDENTIFIER := expr TO expr stmt_list END
expr -> IDENTIFIER
expr -> NUMBER
expr -> STRING
```

First and Follow set:

FIRST SETS:				FOLLOW SETS:			
Non-Terminal	First Set			Non-Terminal	Follow Set		
program	BEGIN			program	\$		
stmt_list	PRINT, INTEGER, REAL, STRING, FOR, END			stmt_list	END		
declaration	INTEGER, REAL, STRING			declaration	PRINT, INTEGER, REAL, STRING, FOR, END		
var_list	IDENTIFIER			var_list	;		
var_list_tail	,, ϵ			var_list_tail	;		
assignment	IDENTIFIER			assignment	PRINT, INTEGER, REAL, STRING, FOR, END		
for_loop	FOR			for_loop	PRINT, INTEGER, REAL, STRING, FOR, END		
expr	IDENTIFIER, NUMBER, STRING			expr	;,)		
type	INTEGER, REAL, STRING			type	IDENTIFIER		

Parsing Table:

PARSING TABLE:																
Non-Terminal	BEGIN	PRINT	INT	REAL	STR	FOR	END	ID	:=	TO	NUM	STR	;	,	\$	
Non-Terminal	BEGIN	PRINT	INT	REAL	STR	FOR	END	ID	:=	TO	NUM	STR	;	,	\$	
program	BEGIN s1 END	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
stmt_list	-	s s1	s s1	s s1	s s1	s s1	ϵ	-	-	-	-	-	-	-	-	-
stmt	-	PRINT e;	decl	decl	decl	for	-	asgn	-	-	-	-	-	-	-	-
decl	-	-	t vl;	t vl;	t vl;	-	-	-	-	-	-	-	-	-	-	-
type	-	-	INT	REAL	STR	-	-	-	-	-	-	-	-	-	-	-
var_list	-	-	-	-	-	-	-	id vt	-	-	-	-	-	-	-	-
var_list_tail	-	-	-	-	-	-	-	-	-	-	-	-	ϵ	,id vt	-	-
assignment	-	-	-	-	-	-	-	id:=e;	-	-	-	-	-	-	-	-
for_loop	-	-	-	-	-	FOR id:=e TO e s1	-	-	-	-	-	-	-	-	-	-
expr	-	-	-	-	-	-	-	id	-	-	num	str	-	-	-	-

Parsing Action:

FULL PARSING ACTIONS:							
Step	Stack	Input Symbol	Action				
1	\$		Start parsing	39	\$	STRING	Apply: declaration $\rightarrow$ type var_list ;
2	\$	BEGIN	Apply: program $\rightarrow$ BEGIN stmt_list END	40	\$	STRING	Apply: type $\rightarrow$ STRING
3	\$ BEGIN	BEGIN	Match 'BEGIN'	41	\$ STRING	STRING	Match 'STRING'
4	\$	PRINT	Apply: stmt_list $\rightarrow$ stmt stmt_list	42	\$	X	Apply: var_list $\rightarrow$ IDENTIFIER var_list_tail
5	\$	PRINT	Apply: stmt $\rightarrow$ PRINT expr ;	43	\$ IDENTIFIER	X	Match 'X'
6	\$ PRINT	PRINT	Match 'PRINT'	44	\$	,	Apply: var_list_tail $\rightarrow$, IDENTIFIER var_list_tail
7	\$	"HELLO"	Apply: expr $\rightarrow$ STRING	45	\$,	,	Match ','
8	\$ STRING	"HELLO"	Match '"HELLO"'	46	\$ IDENTIFIER	Y	Match 'Y'
9	\$;	;	Match ';' ;	47	\$	;	Apply: var_list_tail $\rightarrow$ ϵ
10	\$	INTEGER	Apply: stmt_list $\rightarrow$ stmt stmt_list	48	\$;	;	Match ';' ;
11	\$	INTEGER	Apply: stmt $\rightarrow$ declaration	49	\$	A	Apply: stmt_list $\rightarrow$ stmt stmt_list
12	\$	INTEGER	Apply: declaration $\rightarrow$ type var_list ;	50	\$	A	Apply: stmt $\rightarrow$ assignment
13	\$	INTEGER	Apply: type $\rightarrow$ INTEGER	51	\$	A	Apply: assignment $\rightarrow$ IDENTIFIER := expr ;
14	\$ INTEGER	INTEGER	Match 'INTEGER'	52	\$ IDENTIFIER	A	Match 'A'
15	\$	A	Apply: var_list $\rightarrow$ IDENTIFIER var_list_tail	53	\$:=	:=	Match ':='
16	\$ IDENTIFIER	A	Match 'A'	54	\$	2	Apply: expr $\rightarrow$ NUMBER
17	\$	,	Apply: var_list_tail $\rightarrow$, IDENTIFIER var_list_tail	55	\$ NUMBER	2	Match '2'
18	\$,	,	Match ',' ;	56	\$;	;	Match ';' ;
19	\$ IDENTIFIER	B	Match 'B'	57	\$	B	Apply: stmt_list $\rightarrow$ stmt stmt_list
20	\$	,	Apply: var_list_tail $\rightarrow$, IDENTIFIER var_list_tail	58	\$	B	Apply: stmt $\rightarrow$ assignment
21	\$,	,	Match ',' ;	59	\$	B	Apply: assignment $\rightarrow$ IDENTIFIER := expr ;
22	\$ IDENTIFIER	C	Match 'C'	60	\$ IDENTIFIER	B	Match 'B'
23	\$	;	Apply: var_list_tail $\rightarrow$ ϵ	61	\$:=	:=	Match ':='
24	\$;	;	Match ';' ;	62	\$	4	Apply: expr $\rightarrow$ NUMBER
25	\$	REAL	Apply: stmt_list $\rightarrow$ stmt stmt_list	63	\$ NUMBER	4	Match '4'
26	\$	REAL	Apply: stmt $\rightarrow$ declaration	64	\$;	;	Match ';' ;
27	\$	REAL	Apply: declaration $\rightarrow$ type var_list ;	65	\$	C	Apply: stmt_list $\rightarrow$ stmt stmt_list
28	\$	REAL	Apply: type $\rightarrow$ REAL	66	\$	C	Apply: stmt $\rightarrow$ assignment
29	\$ REAL	REAL	Match 'REAL'	67	\$	C	Apply: assignment $\rightarrow$ IDENTIFIER := expr ;
30	\$	D	Apply: var_list $\rightarrow$ IDENTIFIER var_list_tail	68	\$ IDENTIFIER	C	Match 'C'
31	\$ IDENTIFIER	D	Match 'D'	69	\$:=	:=	Match ':='
32	\$	,	Apply: var_list_tail $\rightarrow$, IDENTIFIER var_list_tail	70	\$	6	Apply: expr $\rightarrow$ NUMBER
33	\$,	,	Match ',' ;	71	\$ NUMBER	6	Match '6'
34	\$ IDENTIFIER	E	Match 'E'	72	\$;	;	Match ';' ;
35	\$	;	Apply: var_list_tail $\rightarrow$ ϵ	73	\$	D	Apply: stmt_list $\rightarrow$ stmt stmt_list
36	\$;	;	Match ';' ;	74	\$	D	Apply: stmt $\rightarrow$ assignment
37	\$	STRING	Apply: stmt_list $\rightarrow$ stmt stmt_list	75	\$	D	Apply: assignment $\rightarrow$ IDENTIFIER := expr ;
38	\$	STRING	Apply: stmt $\rightarrow$ declaration	76	\$ IDENTIFIER	D	Match 'D'
				77	\$:=	:=	Match ':='
				78	\$	-3.56E-8	Apply: expr $\rightarrow$ NUMBER

79	\$	NUMBER	-3.56E-8	Match '-3.56E-8'
80	\$	;	;	Match ';'
81	\$		E	Apply: stmt_list → stmt stmt_list
82	\$		E	Apply: stmt → assignment
83	\$		E	Apply: assignment → IDENTIFIER := expr ;
84	\$	IDENTIFIER	E	Match 'E'
85	\$	:=	:=	Match ':='
86	\$		4.567	Apply: expr → NUMBER
87	\$	NUMBER	4.567	Match '4.567'
88	\$	;	;	Match ';'
89	\$		X	Apply: stmt_list → stmt stmt_list
90	\$		X	Apply: stmt → assignment
91	\$		X	Apply: assignment → IDENTIFIER := expr ;
92	\$	IDENTIFIER	X	Match 'X'
93	\$	:=	:=	Match ':='
94	\$		"text1"	Apply: expr → STRING
95	\$	STRING	"text1"	Match '"text1"'
96	\$	;	;	Match ';'
97	\$		Y	Apply: stmt_list → stmt stmt_list
98	\$		Y	Apply: stmt → assignment
99	\$		Y	Apply: assignment → IDENTIFIER := expr ;
100	\$	IDENTIFIER	Y	Match 'Y'
101	\$	:=	:=	Match ':='
102	\$		"hello there"	Apply: expr → STRING
103	\$	STRING	"hello there"	Match '"hello there"'
104	\$	;	;	Match ';'
105	\$		FOR	Apply: stmt_list → stmt stmt_list
106	\$		FOR	Apply: stmt → for_loop
107	\$		FOR	Apply: for_loop → FOR IDENTIFIER := expr TO expr stmt_list END
108	\$	FOR	FOR	Match 'FOR'
109	\$	IDENTIFIER	I	Match 'I'
110	\$	:=	:=	Match ':='
111	\$		1	Apply: expr → NUMBER
112	\$	NUMBER	1	Match '1'
113	\$	TO	TO	Match 'TO'
114	\$		5	Apply: expr → NUMBER
115	\$	NUMBER	5	Match '5'
116	\$		PRINT	Apply: stmt_list → stmt stmt_list
117	\$		PRINT	Apply: stmt → PRINT expr ;
118	\$	PRINT	PRINT	Match 'PRINT'
119	\$		"Strings are [X] and [Y]"	Apply: expr → STRING
120	\$	STRING	"Strings are [X] and [Y]"	Match '"Strings are [X] and [Y]"'
121	\$	;	;	Match ';'
122	\$		END	Apply: stmt_list → ε
123	\$		END	Apply: stmt_list → ε
124	\$	END	END	Match 'END'
125	\$		\$	Parsing completed successfully!

Dept of CS Parsing completed successfully with all statements processed!

CONCLUSION

In this project, we learned how a compiler works by breaking down a program into smaller parts using lexical analysis and parsing. The lexical analyzer successfully identified different parts of the code like keywords, variables, numbers, and symbols. These were stored properly in symbol tables.

We then used grammar rules and created a parsing table to check if the given input follows the correct syntax. Specifically, we implemented the **LL(1) parsing technique**, which allowed the parser to analyze the code from left to right using one lookahead symbol, making the parsing process efficient and predictive.

This helped us understand how a parser processes the code step-by-step. Overall, this project helped us understand the basic steps involved in compiling a program. It was a great experience in applying what we learned in compiler design.

REFERENCES

- <https://www.javatpoint.com/compiler-tutorial>
- https://www.tutorialspoint.com/compiler_design/index.htm
- <https://www.geeksforgeeks.org/compiler-design-tutorials/>
- <https://docs.python.org/3/library/re.html>
- <https://pypi.org/project/tabulate/>